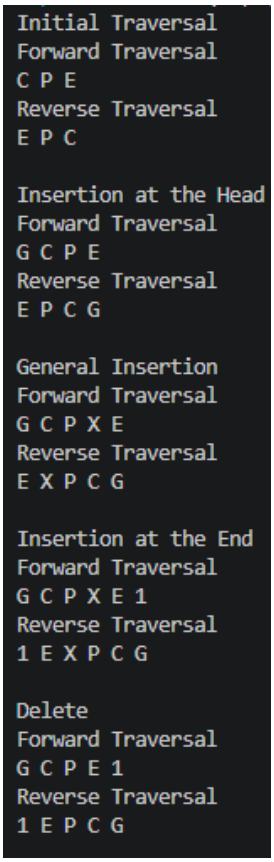
COMPUTER ENGINEERING

QUEZON CITY

| | | | |
|---|---|---|---|
| **COURSE CODE/TITLE:** | CpE010-DSA | **SECTION:** | CpE21S1 |
| **DATE PERFORMED:** | 7-23-26 | **DATE SUBMITTED:** | 7-27-2026 |
| **NAME:** | Mauro, Ryan Lewis B. | **INSTRUCTOR:** | Engr. Jimlord Quejado |

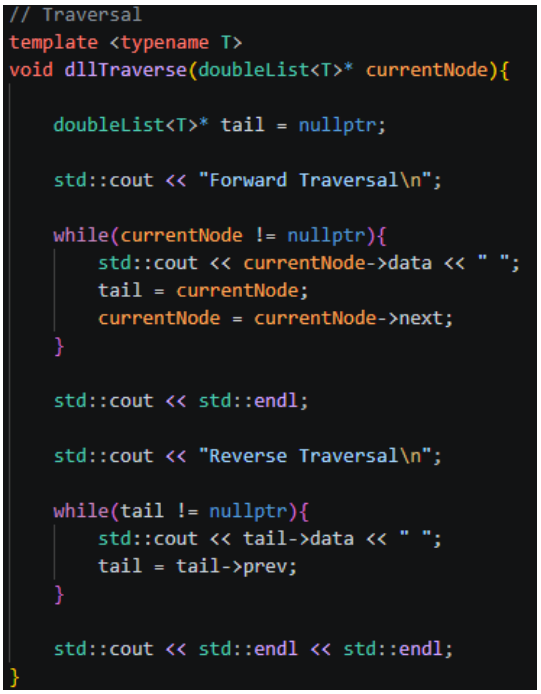
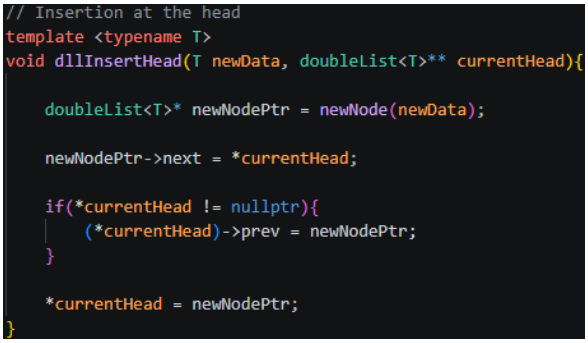
**ACTIVITY NO. 3.1 [Linked list]**

**1. PROCEDURE OUTPUT**

| | |
|---|---|
| **Screenshot** |  |
| **Discussion** | The program successfully demonstrated the basic operations of a doubly linked list. It first created the initial list containing the values C, P, and E, then displayed them in both forward and reverse order, showing that the next and prev pointers were linked correctly. After inserting G at the head, the new node became the first element of the list. The general insertion operation then added X after P, proving that a new node can be inserted at any position in the list |

| | |
|---|---|
| | while maintaining the correct links. Next, inserting 1 at the end showed that the list can also be expanded from the tail. Finally, the delete operation removed X from the list, and the forward and reverse traversals confirmed that the remaining nodes were still connected properly. Overall, the output shows that all insertion and deletion operations worked as expected while preserving the integrity of the doubly linked list. |

**Table 3-1. Output of Initial/Simple Implementation**

| Operation | Screenshot |
|---|---|
| Traversal |  |
| Insertion at the head |  |

```cpp
// Traversal
template <typename T>
void dllTraverse(doubleList<T>* currentNode){

    doubleList<T>* tail = nullptr;

    std::cout << "Forward Traversal\n";

    while(currentNode != nullptr){
        std::cout << currentNode->data << " ";
        tail = currentNode;
        currentNode = currentNode->next;
    }

    std::cout << std::endl;

    std::cout << "Reverse Traversal\n";

    while(tail != nullptr){
        std::cout << tail->data << " ";
        tail = tail->prev;
    }

    std::cout << std::endl << std::endl;
}
```

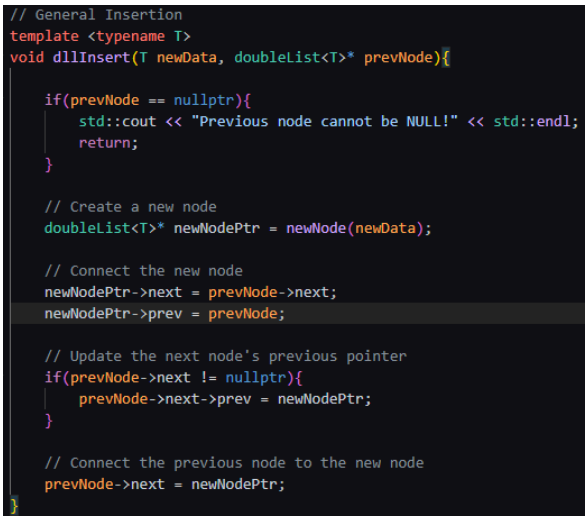
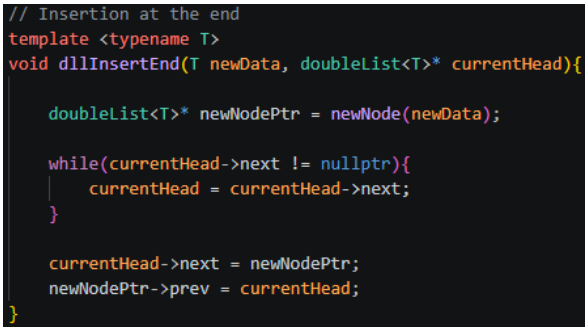
```cpp
// Insertion at the head
template <typename T>
void dllInsertHead(T newData, doubleList<T>** currentHead){

    doubleList<T>* newNodePtr = newNode(newData);

    newNodePtr->next = *currentHead;

    if(*currentHead != nullptr){
        (*currentHead)->prev = newNodePtr;
    }

    *currentHead = newNodePtr;
}
```

| | |
|---|---|
| Insertion at any part of the list |  |
| Insertion at the end |  |
| Deletion of a node |  |

```
// General Insertion
template <typename T>
void dllInsert(T newData, doubleList<T>* prevNode){

    if(prevNode == nullptr){
        std::cout << "Previous node cannot be NULL!" << std::endl;
        return;
    }

    // Create a new node
    doubleList<T>* newNodePtr = newNode(newData);

    // Connect the new node
    newNodePtr->next = prevNode->next;
    newNodePtr->prev = prevNode;

    // Update the next node's previous pointer
    if(prevNode->next != nullptr){
        prevNode->next->prev = newNodePtr;
    }

    // Connect the previous node to the new node
    prevNode->next = newNodePtr;
}
```

```
// Insertion at the end
template <typename T>
void dllInsertEnd(T newData, doubleList<T>* currentHead){

    doubleList<T>* newNodePtr = newNode(newData);

    while(currentHead->next != nullptr){
        currentHead = currentHead->next;
    }

    currentHead->next = newNodePtr;
    newNodePtr->prev = currentHead;
}
```

```
// Delete
template <typename T>
void dllDelete(doubleList<T>** currentHead, doubleList<T>* deleteNode){

    if(*currentHead == nullptr || deleteNode == nullptr){
        return;
    }

    if(*currentHead == deleteNode){
        *currentHead = deleteNode->next;
    }

    if(deleteNode->prev != nullptr){
        deleteNode->prev->next = deleteNode->next;
    }

    if(deleteNode->next != nullptr){
        deleteNode->next->prev = deleteNode->prev;
    }

    delete deleteNode;
}
```

**Table 3-2. Code for the List Operations**

**a.**

**Source Code**

```
// Creating a node
template <typename T>
doubleList<T>* newNode(T newData){

    doubleList<T>* newNode = new doubleList<T>;

    newNode->data = newData;
    newNode->next = nullptr;
    newNode->prev = nullptr;

    return newNode;
}
```

**Console**

```
Initial Traversal
Forward Traversal
C P E
Reverse Traversal
E P C
```

**b.**

**Source Code**

```
// Traversal
template <typename T>
void dllTraverse(doubleList<T>* currentNode){

    doubleList<T>* tail = nullptr;

    std::cout << "Forward Traversal\n";

    while(currentNode != nullptr){
        std::cout << currentNode->data << " ";
        tail = currentNode;
        currentNode = currentNode->next;
    }

    std::cout << std::endl;

    std::cout << "Reverse Traversal\n";

    while(tail != nullptr){
        std::cout << tail->data << " ";
        tail = tail->prev;
    }

    std::cout << std::endl << std::endl;
}
```

**Console**

```
Forward Traversal
C P E
Reverse Traversal
E P C
```

**c.**

**Source Code**

```
// Insertion at the head
template <typename T>
void dllInsertHead(T newData, doubleList<T>** currentHead){

    doubleList<T>* newNodePtr = newNode(newData);

    newNodePtr->next = *currentHead;

    if(*currentHead != nullptr){
        (*currentHead)->prev = newNodePtr;
    }

    *currentHead = newNodePtr;
}
```

**Console**

```
Insertion at the Head
Forward Traversal
G C P E
Reverse Traversal
E P C G
```

**d.**

**Source Code**

```
// General Insertion
template <typename T>
void dllInsert(T newData, doubleList<T>* prevNode){

    if(prevNode == nullptr){
        std::cout << "Previous node cannot be NULL!" << std::endl;
        return;
    }

    // Create a new node
    doubleList<T>* newNodePtr = newNode(newData);

    // Connect the new node
    newNodePtr->next = prevNode->next;
    newNodePtr->prev = prevNode;

    // Update the next node's previous pointer
    if(prevNode->next != nullptr){
        prevNode->next->prev = newNodePtr;
    }

    // Connect the previous node to the new node
    prevNode->next = newNodePtr;
}
```

**Console**

```
General Insertion
Forward Traversal
G C P X E
Reverse Traversal
E X P C G
```

e.

**Source Code**

```
// Insertion at the end
template <typename T>
void dllInsertEnd(T newData, doubleList<T>* currentHead){

    doubleList<T>* newNodePtr = newNode(newData);

    while(currentHead->next != nullptr){
        currentHead = currentHead->next;
    }

    currentHead->next = newNodePtr;
    newNodePtr->prev = currentHead;
}
```

**Console**

```
Insertion at the End
Forward Traversal
G C P X E 1
Reverse Traversal
1 E X P C G
```

f.

**Source Code**

```
// Delete
template <typename T>
void dllDelete(T findData, doubleList<T>** head){

    // Check if the list is empty
    if(*head == nullptr) return;

    doubleList<T>* currentNode = *head;

    // Search for the node
    while(currentNode != nullptr && currentNode->data != findData){
        currentNode = currentNode->next;
    }

    // If data is not found
    if(currentNode == nullptr) return;

    // Delete the head node
    if(currentNode->prev == nullptr){
        *head = currentNode->next;

        if(*head != nullptr){
            (*head)->prev = nullptr;
        }
    }
    else{
        currentNode->prev->next = currentNode->next;

        if(currentNode->next != nullptr){
            currentNode->next->prev = currentNode->prev;
        }
    }

    delete currentNode;
}
```

| | |
|---|---|
| **Console** | Delete<br>Forward Traversal<br>G C P E 1<br>Reverse Traversal<br>1 E P C G |

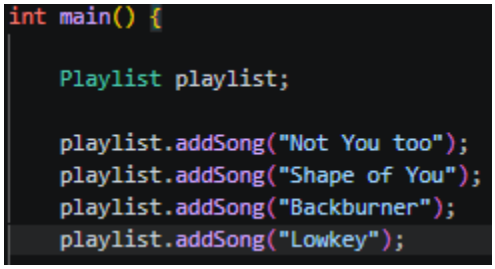
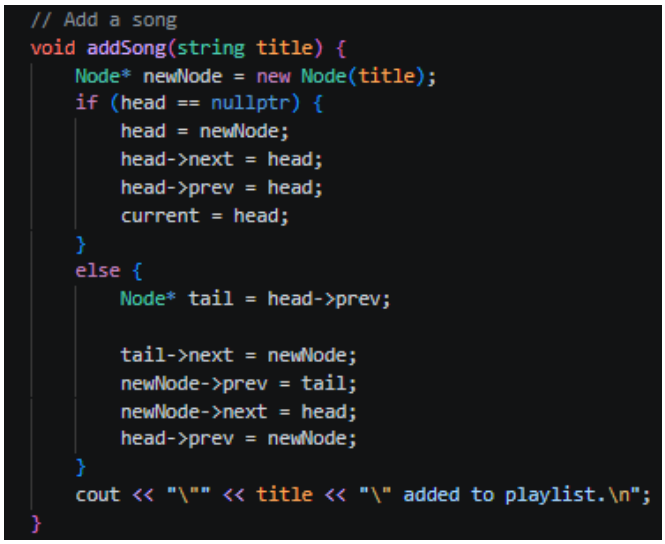
**Table 3-3. Code and Analysis for Singly Linked Lists**

| Screenshot(s) | Analysis |
|---|---|
| `void dllTraverse(doubleList<T>* currentNode){` | This operation displays the elements of the doubly linked list in both forward and reverse directions. It verifies that both next and prev pointers are working correctly. |
| `void dllInsertHead(T newData, doubleList<T>** currentHead){` | This operation inserts a new node at the beginning of the doubly linked list. The new node becomes the head, and the previous pointer of the old head is updated to maintain the connection between nodes. |
| `void dllInsert(T newData, doubleList<T>* prevNode){` | This operation allows inserting a new node at any position in the list. It updates both the next and prev pointers so that the new node is properly connected with the previous and next nodes. |
| `void dllInsertEnd(T newData, doubleList<T>* currentHead){` | This operation adds a new node at the end of the list. The function traverses until the last node and connects the new node using the next pointer and sets its prev pointer to the previous last node. |
| `void dllDelete(T findData, doubleList<T>** head){` | This operation removes a specific node from the doubly linked list by searching for the given data. It updates the neighboring nodes' pointers and releases the memory of the deleted node. |

**Table 3-4. Modified Operations for Doubly Linked Lists**

## 2. ANSWERS TO SUPPLEMENTAL ACTIVITY

**ILO B: Solve given problems utilizing linked lists in C++**
**Problem Title: Implementing a Song Playlist using Linked List**
**Source: Packt Publishing**

| Funtion | Screenshot |
|---|---|
| Create a playlist using multiple songs. |  |
| Add songs to the playlist. |  |

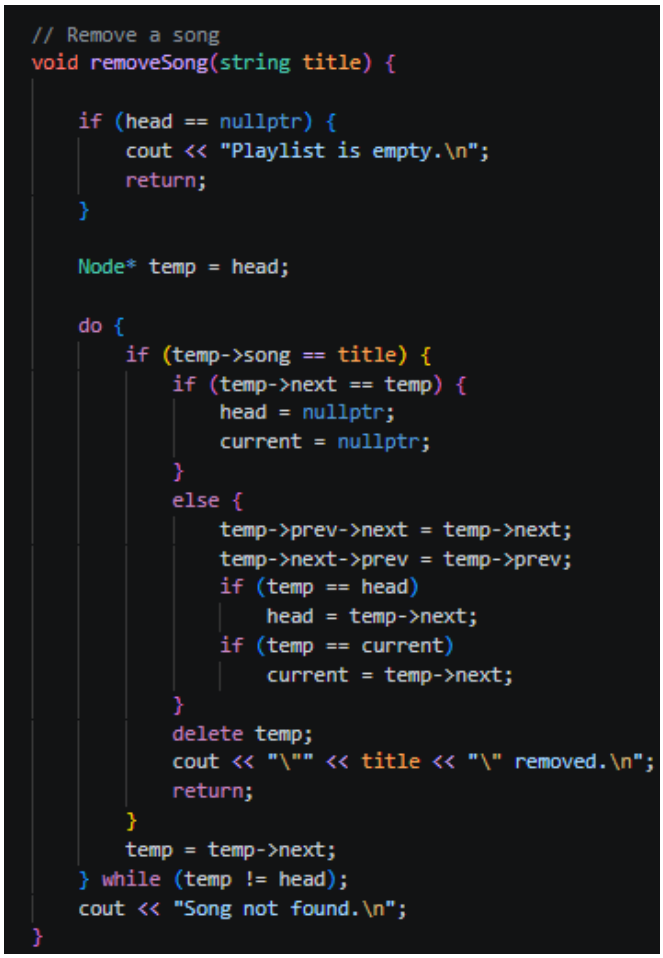
```cpp
int main() {

    Playlist playlist;

    playlist.addSong("Not You too");
    playlist.addSong("Shape of You");
    playlist.addSong("Backburner");
    playlist.addSong("Lowkey");
```

```cpp
// Add a song
void addSong(string title) {
    Node* newNode = new Node(title);
    if (head == nullptr) {
        head = newNode;
        head->next = head;
        head->prev = head;
        current = head;
    }
    else {
        Node* tail = head->prev;

        tail->next = newNode;
        newNode->prev = tail;
        newNode->next = head;
        head->prev = newNode;
    }
    cout << "\"" << title << "\" added to playlist.\n";
}
```

| | |
|---|---|
| Remove a song from the playlist. |  |
| Play songs in a loop (for this activity, we will print all the songs once). |  |

```
// Remove a song
void removeSong(string title) {

    if (head == nullptr) {
        cout << "Playlist is empty.\n";
        return;
    }

    Node* temp = head;

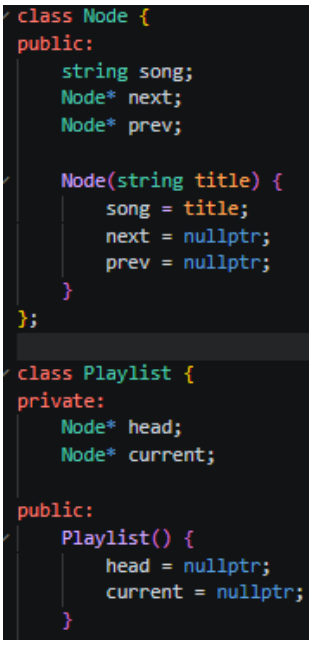
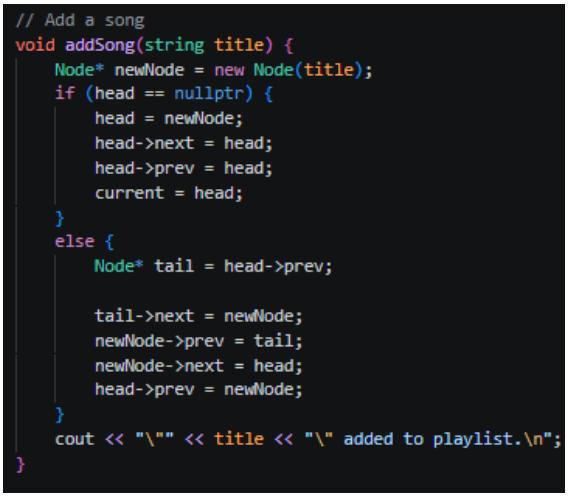
    do {
        if (temp->song == title) {
            if (temp->next == temp) {
                head = nullptr;
                current = nullptr;
            }
            else {
                temp->prev->next = temp->next;
                temp->next->prev = temp->prev;
                if (temp == head)
                    head = temp->next;
                if (temp == current)
                    current = temp->next;
            }
            delete temp;
            cout << "\"" << title << "\" removed.\n";
            return;
        }
        temp = temp->next;
    } while (temp != head);
    cout << "Song not found.\n";
}
```

```
// It will display playlist
void playAll() {

    if (head == nullptr) {
        cout << "Playlist is empty.\n";
        return;
    }

    Node* temp = head;
    cout << "\nPlaylist:\n";
    do {
        cout << temp->song << endl;
        temp = temp->next;
    } while (temp != head);
}
```

**Table 3.5. functions supported for Supplementary Task**

| Problem | Screenshot |
| --- | --- |
| Design the basic structure that supports circular data representation. |  |
| After that, implement the insert functions in structure. |  |

```
class Node {
public:
    string song;
    Node* next;
    Node* prev;

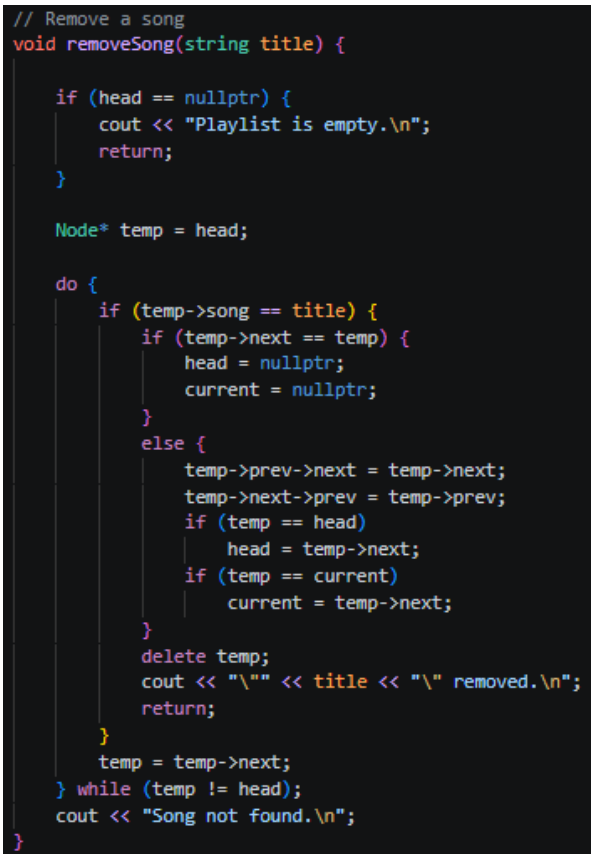
    Node(string title) {
        song = title;
        next = nullptr;
        prev = nullptr;
    }
};

class Playlist {
private:
    Node* head;
    Node* current;

public:
    Playlist() {
        head = nullptr;
        current = nullptr;
    }
```

```
// Add a song
void addSong(string title) {
    Node* newNode = new Node(title);
    if (head == nullptr) {
        head = newNode;
        head->next = head;
        head->prev = head;
        current = head;
    }
    else {
        Node* tail = head->prev;

        tail->next = newNode;
        newNode->prev = tail;
        newNode->next = head;
        head->prev = newNode;
    }
    cout << "\"" << title << "\" added to playlist.\n";
}
```

| | |
|---|---|
| Implement the delete functions in the structure. |  |
| Implement a function for traversing the playlist. |  |

```
// Remove a song
void removeSong(string title) {

    if (head == nullptr) {
        cout << "Playlist is empty.\n";
        return;
    }

    Node* temp = head;

    do {
        if (temp->song == title) {
            if (temp->next == temp) {
                head = nullptr;
                current = nullptr;
            }
            else {
                temp->prev->next = temp->next;
                temp->next->prev = temp->prev;
                if (temp == head)
                    head = temp->next;
                if (temp == current)
                    current = temp->next;
            }
            delete temp;
            cout << "\"" << title << "\" removed.\n";
            return;
        }
        temp = temp->next;
    } while (temp != head);
    cout << "Song not found.\n";
}
```

```
// It will display playlist
void playAll() {

    if (head == nullptr) {
        cout << "Playlist is empty.\n";
        return;
    }

    Node* temp = head;
    cout << "\nPlaylist:\n";
    do {
        cout << temp->song << endl;
        temp = temp->next;
    } while (temp != head);
}
```

**Table 3.6. Solving a problem for Supplementary Task**

Conclusion: In this activity, I learned how circular linked lists work and how they can be applied in real-world situations such as creating a music playlist. I understood that unlike a normal linked list, a circular linked list connects the last node back to the first node, allowing continuous traversal. The procedure helped me learn how to create the node structure, implement insertion and deletion functions, and properly traverse the playlist while

maintaining the correct links between nodes. The supplementary activity made the concept easier to understand because it showed how linked lists are useful for applications that require repeated movement through data. I think I performed well in this activity because I was able to apply the concepts of pointers and nodes to create a working playlist system. However, I still need to improve my skills in managing pointers, especially when deleting nodes and updating connections. Practicing more linked list problems will help me gain more confidence and improve my understanding of data structures.

## 3. ARTIFICIAL INTELLIGENCE (AI) USE DISCLOSURE STATEMENT

I hereby declare that artificial intelligence (AI) tools were used, if applicable, in the preparation of this academic work. The following indicates the nature and extent of AI assistance:

**Tools Used:** **Chatgpt, Grammarly**

*Examples: ChatGPT, Microsoft Copilot, Grammarly, QuillBot, etc.*

**Purpose of Use:** **Grammar correction code assistance and asking for idea**

*Examples: grammar correction, idea generation, formatting, summarization, code assistance, explanation of concepts, etc.*

**Extent of Use:** **Grammar correction and ask how a circular linked list helps in creating a music playlist. Also ask for code assistance since I encountered a lot of error**

*Explain how AI was used and confirm that it did not replace original thinking, analysis, authorship, or completion of the required work.*

By submitting this activity, I affirm that my use of AI complies with T.I.P.'s AI usage policy and does not exceed the permitted limits for similarity, automated content generation, or academic assistance.

I understand that any false, incomplete, or misleading disclosure may be subject to investigation in accordance with due process and may result in sanctions for violation of existing T.I.P. policies.

Lab activities, templates, and related instructional materials shall not be reuploaded, reproduced, distributed, or shared with external organizations or third parties without the prior express written permission of the Technological Institute of the Philippines.